STUDIFYX: AN INTEGRATED EXPLAINABLE AI PLATFORM FOR ACADEMIC GUIDANCE, CAREERS AND WELLBEING

INDIVIDUAL THESIS

Rusiru L.W.S
IT22220288

INDIVIDUAL COMPONENT:
FUTUREDREAM - AN AI-POWERED CAREER GUIDANCE SYSTEM FOR DEGREE AND UNIVERSITY RECOMMENDATION FOR SRI LANKAN A/L STUDENTS

Report submitted in partial fulfillment of the requirements for the Bachelor of Science (Hons) in Information Technology

Department of Information Technology

Sri Lanka Institute of Information Technology
Sri Lanka

October 2026

DECLARATION

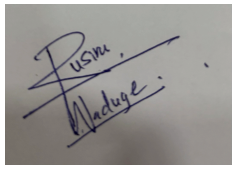
I declare that this is my own work and this dissertation does not incorporate without acknowledgement any material previously submitted for a Degree or Diploma in any other University or institute of higher learning and to the best of my knowledge and belief it does not contain any material previously published or written by another person except where the acknowledgement is made in the text.

Also, I hereby grant to Sri Lanka Institute of Information Technology, the non-exclusive right to reproduce and distribute my dissertation, in whole or in part in print, electronic or other medium. I retain the right to use this content in whole or part in future works (such as articles or books).

Academic integrity and AI disclosure. I have disclosed all use of generative AI and other automated tools in accordance with the module policy. I remain responsible for the accuracy, originality, citations, code, data, analyses, and conclusions in this dissertation.

Student name: Rusiru L.W.S

Student ID: IT22220288

Signature:  Date: 09/10/2026

Supervisor certification. The above candidate has carried out research for the bachelor's degree Dissertation under my supervision.

Role and name	Signature	Date
Supervisor: Ms.Thilini Jayalath		
Co-supervisor: Ms.Jenny Krishara		

Abstract

Sri Lankan Advanced Level (A/L) students must choose a degree programme and university within a competitive, Z-score-based admission system with district quotas, usually with little personalised or data-driven guidance. This dissertation presents the design, implementation and evaluation of FutureDream, an AI-powered career guidance module that provides personalised degree recommendations and university matching for Sri Lankan A/L students. The module combines a Random Forest classifier that predicts a suitable degree from the student's Z-score, A/L stream, subject results, district and personality traits; a rule-based university recommender that estimates admission probability for government universities using Z-score cut-offs and district quotas and lists matching private universities; and a backward-chaining engine that starts from the student's dream job and traces the degrees, skills and A/L subjects required to reach it, producing a timeline and gap analysis. The system is delivered as a React Native mobile application with a Flask backend and Firebase for authentication and data persistence. On the test set, the Random Forest model achieved an accuracy of 87.3% and a weighted F1-score of 86.5%, government university admission predictions were 84.2% accurate against historical data, and preliminary testing with 25 students gave an overall satisfaction rating of 4.1 out of 5. The results show that a hybrid of machine learning and rule-based logic, adapted to the Sri Lankan admission context, can provide accessible, explainable and personalised higher-education guidance.

Keywords: career guidance, random forest, backward chaining, university recommendation, Sri Lankan education

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my supervisor, Ms.Thilini Jayalath, and co-supervisor, Ms.Jenny Krishara, for their guidance, constructive feedback and encouragement throughout this research.

I thank the members of the R26-IT-073 project group for their cooperation, the academic counsellors who reviewed the generated career timelines, and the students who took part in the user testing and shared their feedback. I also thank the Sri Lanka Institute of Information Technology for providing the facilities and resources for this work, and my family and friends for their constant support.

TABLE OF CONTENTS

DECLARATION i
Abstract ii
ACKNOWLEDGEMENT iii
TABLE OF CONTENTS iv
LIST OF FIGURES ix
LIST OF TABLES x
LIST OF ABBREVIATIONS xi
LIST OF APPENDICES xii
1 INTRODUCTION 1
1.1 Background and Literature Survey 1
1.1.1 Background 1
1.1.2 Individual component context 1
1.1.3 Motivation 2
1.1.4 Career guidance systems 3
1.1.5 Machine learning in education 3
1.1.6 University recommendation systems 3
1.1.7 Backward chaining in career planning 4
1.1.8 Existing approaches 4

1.2 Research Gap 5
1.3 Research Problem 6
1.3.1 Research questions 7
1.3.2 Hypotheses 7
1.4 Research Objectives 8
1.4.1 Research significance 8
1.4.2 Scope and limitations 9
1.4.3 Structure of the dissertation 9
2 METHODOLOGY 10
2.1 Methodology 10
2.1.1 Research approach 10
2.1.2 Data collection 11
2.1.3 Data pre-processing 11
2.1.4 Model selection 11
2.1.5 Model training configuration 12
2.1.6 Evaluation metrics 13
2.1.7 University matching algorithm 13
2.1.8 District quota integration 14
2.1.9 Backward-chaining design 14
2.1.10 System architecture 15

2.1.11 Frontend design 16
2.1.12 Backend design 17
2.1.13 Database design 18
2.1.14 Security design 18
2.1.15 User interface design 19
2.2 Commercialization Aspects of the Product 19
2.2.1 Value proposition and target users 19
2.2.2 Market and competitive position 19
2.2.3 Business and sustainability model 19
2.2.4 Scalability, intellectual property and compliance 20
2.2.5 Contribution to the Sustainable Development Goals 20
2.3 Testing and Implementation 21
2.3.1 Technology stack 21
2.3.2 Recommendation form 21
2.3.3 Future Dream Advisor 22
2.3.4 API service integration 23
2.3.5 Backend and model implementation 24
2.3.6 Integration challenges and solutions 25
2.3.7 Testing 26
2.3.8 Deployment 27

3 RESULTS AND DISCUSSION 28
3.1 Results 28
3.1.1 Model performance 28
3.1.2 Feature importance 28
3.1.3 Confidence score distribution 30
3.1.4 University recommendation accuracy 31
3.1.5 User testing 31
3.1.6 Backward-chaining effectiveness 31
3.1.7 System performance 32
3.2 Research Findings 33
3.3 Discussion 34
3.3.1 Comparison with existing approaches 34
3.3.2 Lessons learned 34
3.3.3 Limitations 34
3.3.4 Implications 35
3.4 Summary of Student's Contribution 35
4 CONCLUSION 36
4.1 Conclusion 36
4.2 Achievement of Objectives 36
4.3 Recommendations and Future Work 37

REFERENCES 38
GLOSSARY 41
APPENDICES 42
Appendix A: System requirements, API specification and subject-stream mapping 42
Appendix B: Academic integrity and AI use disclosure 43

LIST OF FIGURES

Figure 1.1: Overview of the FutureDream module 2
Figure 1.2: Conceptual framework of FutureDream 5
Figure 2.1: Design Science Research phases applied in this work.....9
Figure 2.2: Example of the career knowledge graph for "Software Engineer" 13
Figure 2.3: Three-tier architecture of FutureDream 14
Figure 2.4: Data flow between the mobile app, backend and Firestore 15
Figure 2.5: Recommendation form and results screens.....20
Figure 2.6: Future Dream Advisor screens.....20
Figure 3.1: Feature importance chart.....26
Figure 3.2: Distribution of prediction confidence scores 27

LIST OF TABLES

Table 1.1: Comparison of FutureDream with existing approaches 4
Table 1.2: Research hypotheses 6
Table 1.3: Research objectives 7
Table 2.1: Data sources 9
Table 2.2: Data pre-processing pipeline 10
Table 2.3: Alternative models considered 10
Table 2.4: Training configuration 11
Table 2.5: Node types of the career knowledge graph 13
Table 2.6: Main screens of the mobile application 14
Table 2.7: REST API endpoints 15
Table 2.8: Main backend classes 15
Table 2.9: Proposed revenue model 17
Table 2.10: Contribution to the Sustainable Development Goals 18
Table 2.11: Technology stack 18
Table 2.12: Integration challenges and solutions 23
Table 2.13: Summary of testing activities 23
Table 2.14: Representative functional test cases 23
Table 3.1: Overall performance of the Random Forest model 25
Table 3.2: Performance by A/L stream 25
Table 3.3: Feature importance of the Random Forest model 26
Table 3.4: Distribution of prediction confidence 26
Table 3.5: University recommendation validation 27
Table 3.6: User testing results (n = 25, scale 1-5) 27
Table 3.7: Career coverage of the knowledge graph 28
Table 3.8: Counsellor review of generated timelines 28
Table 3.9: API and mobile application performance 28
Table 3.10: Validation of hypotheses 29
Table 3.11: Summary of the author's individual contribution 31
Table 4.1: Achievement of objectives 33
Table A.1: System requirements 38
Table A.2: Subject-stream mapping 39
Table B.1: AI tool use disclosure 39

LIST OF ABBREVIATIONS

Abbreviation	Description
A/L	Advanced Level
API	Application Programming Interface
CORS	Cross-Origin Resource Sharing
DSR	Design Science Research
F1	Harmonic mean of precision and recall
G.C.E.	General Certificate of Education
ICT	Information and Communication Technology
JSON	JavaScript Object Notation
ML	Machine Learning
O/L	Ordinary Level
PDPA	Personal Data Protection Act
REST	Representational State Transfer
RF	Random Forest
RIASEC	Realistic, Investigative, Artistic, Social, Enterprising, Conventional
RQ	Research Question
TLS	Transport Layer Security
UGC	University Grants Commission
UI	User Interface

LIST OF APPENDICES

Appendix	Description	Page
Appendix A	System requirements, API specification and subject-stream mapping	38
Appendix B	Academic integrity and AI use disclosure	39

1 INTRODUCTION

This chapter introduces FutureDream, the individual component developed in this dissertation. It presents the background and literature survey, identifies the research gap, defines the research problem and sets out the research objectives.

1.1 Background and Literature Survey

1.1.1 Background

The transition from secondary to higher education is one of the most important decisions in a student's academic life. In Sri Lanka the decision is especially complex because state university admission is competitive and is governed by the Z-score ranking system and district-based quotas administered by the University Grants Commission (UGC) [1]. Students come from six A/L streams (Physical Science, Biological Science, Commerce, Arts, Engineering Technology and Bio Systems Technology), each leading to a specific set of degree programmes at government and private universities [1], [2].

Traditional career guidance in Sri Lanka relies heavily on school counsellors, parents and peers. This advice is valuable but often lacks data-driven personalisation and complete information about emerging career paths and new degree programmes. As the job market and the range of programmes change quickly, an information gap has grown between what students need to know and what they can easily find out.

1.1.2 Individual component context

This dissertation reports only on the author's individual component of the R26-IT-073 research project, the **FutureDream** module. Other components of the project are not discussed. FutureDream helps an A/L student answer two questions: "Which degree suits me?" and "Which universities can I realistically enter for that degree?". It also lets the student start from a dream job and see the path that leads to it. Figure 1.1 gives an overview of the module.

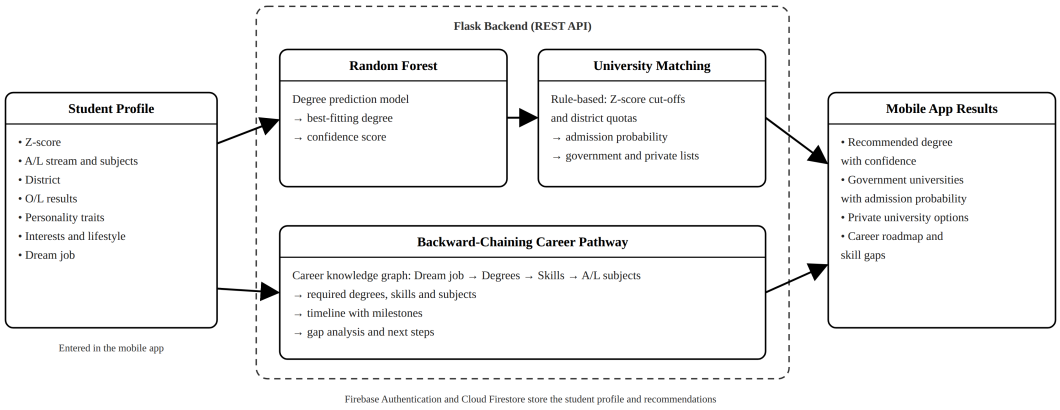


Figure 1.1: Overview of the FutureDream module

For example, a Physical Science student from Colombo with a Z-score of 1.72 who enjoys problem-solving enters her profile and selects "Software Engineer" as her dream job. The module predicts Computer Science as the best-fitting degree, lists government universities with her estimated admission probability for each, lists private universities offering the degree, and shows the skills, subjects and milestones that lead from her current position to the dream job.

1.1.3 Motivation

The research is motivated by five observations:

- **Information asymmetry:** students often lack complete information about available degree programmes, university requirements and career outcomes.
- **Decision complexity:** the interaction between Z-score, stream, subject combination, district and career goal creates a decision space that is difficult to navigate manually.
- **Resource constraints:** professional career counselling is limited, particularly in rural areas, creating unequal access to guidance.
- **Technology gap:** career guidance systems exist internationally, but few are tailored to Sri Lanka's Z-score and district quota system.

- **Data-driven potential:** historical data and machine learning offer an opportunity for more accurate and personalised recommendations.

1.1.4 Career guidance systems

Early computer-assisted career guidance systems relied on rule-based matching of interests to occupations using taxonomies such as Holland's RIASEC model [3]. They supported structured career exploration but did not personalise advice using academic performance or contextual constraints. Social Cognitive Career Theory adds that self-efficacy, outcome expectations and contextual supports and barriers shape academic and career choices [4], which supports considering a student's context as well as interests. Modern systems follow four broad approaches: knowledge-based systems that encode counsellors' expertise, data-driven systems that analyse graduate outcomes, machine learning systems that classify or cluster students, and hybrid systems that combine these for better accuracy and interpretability.

1.1.5 Machine learning in education

Educational data mining applies machine learning to student performance prediction, personalised learning, course recommendation and adaptive assessment [5], [6]. Random Forest classifiers [7] are widely used in this area because they handle mixed numerical and categorical data, resist overfitting through ensembling, produce class probabilities, and provide feature-importance rankings that help explain predictions [8].

1.1.6 University recommendation systems

University and course recommenders typically use collaborative filtering, content-based filtering or hybrid approaches [9]. Most were developed for international contexts and do not account for centralised, merit-based admission, regional quota systems, stream-specific subject requirements, or the distinction between government and private institutions. The Sri Lankan UGC admission process requires a specialised approach [1].

1.1.7 Backward chaining in career planning

Backward chaining is a goal-driven inference strategy from expert systems that starts from a goal and searches for the conditions that support it [10], [11]. Applied to career guidance, it starts from the desired career and identifies the necessary degrees, skills and subjects. This makes the pathway to an aspirational career explicit, reveals skill gaps, produces an actionable roadmap and helps motivate students by making the goal concrete.

1.1.8 Existing approaches

Table 1.1 compares FutureDream with existing approaches to career guidance available to Sri Lankan students.

Table 1.1: Comparison of FutureDream with existing approaches

Aspect	Traditional counselling	UGC handbook / cut-off look-up	International recommenders	FutureDream
Personalisation	Low (generic advice)	None	Medium (interests, grades)	High (ML-based profile)
Sri Lankan Z-score and district quotas	Partly (counsellor knowledge)	Yes (static data)	No	Yes (admission probability)
Starts from career goal	Sometimes	No	Rarely	Yes (backward chaining)
Accessibility	Limited by counsellor availability	Online / print	Web	High (mobile app)
Update frequency	Low (manual)	Yearly	Varies	High (configurable data)
Explainability	High (human explanation)	Not applicable	Low	Medium (hybrid approach)
Scalability	Low (human-dependent)	High	High	High (automated)

1.2 Research Gap

Despite existing work in career guidance and educational recommendation, the literature survey identified five gaps:

- **Contextual adaptation:** little research adapts recommendation systems to national admission policies such as Sri Lanka's Z-score ranking and district quotas.
- **Multi-modal integration:** academic performance, personality traits, interests and lifestyle preferences are rarely combined in one recommendation process.
- **Mobile accessibility:** most systems are web-based, whereas mobile-first delivery is more appropriate for students in developing regions.
- **Explainability:** machine learning recommenders often trade interpretability for accuracy, reducing user trust [12].
- **Continuous learning:** few systems include mechanisms to improve recommendations from user feedback over time.

FutureDream addresses these gaps with a context-aware, multi-modal, mobile-first system that combines a machine learning model with interpretable rule-based components. Figure 1.2 shows the conceptual framework.

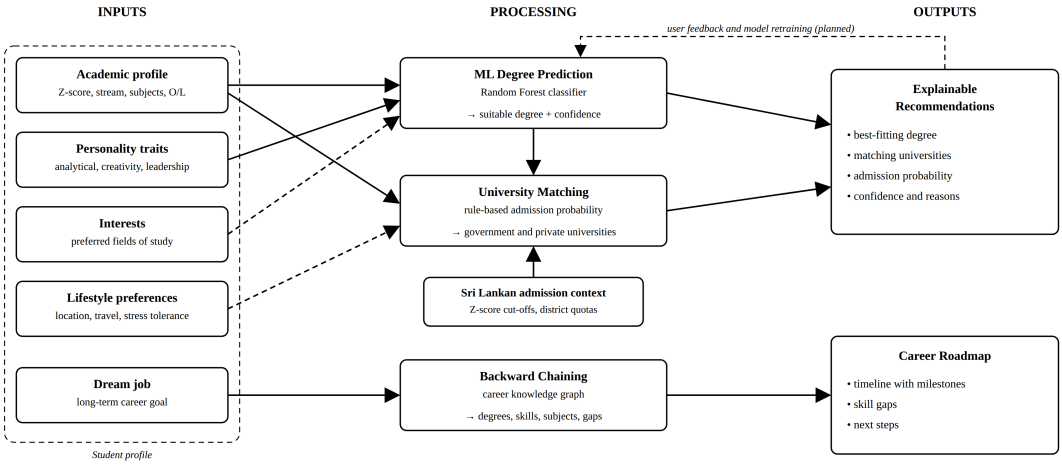


Figure 1.2: Conceptual framework of FutureDream

1.3 Research Problem

Sri Lankan A/L students face significant difficulty in making informed higher-education decisions because of:

- **A complex decision space:** six A/L streams, more than 50 subject combinations, more than 15 universities, more than 100 degree programmes and district-specific admission criteria combine into a very large number of possibilities.
- **Fragmented information:** information on programmes, requirements, career outcomes and admission statistics is spread across UGC publications, university brochures, counsellors and peers.
- **Lack of personalisation:** generic guidance does not consider individual performance, personality, interests and lifestyle.
- **Limited resources:** professional counselling is scarce, especially in rural areas.
- **A dynamic environment:** new programmes appear, admission criteria change and job-market demands evolve, so static guidance quickly becomes outdated.

The research problem is therefore: **how can Sri Lankan A/L students be given personalised, context-aware and explainable degree and university**

recommendations, together with a clear pathway to their career goal, through an accessible mobile application?

1.3.1 Research questions

- **RQ1:** How can machine learning be applied to predict suitable degree programmes for Sri Lankan A/L students from their academic profile, personality traits and career aspirations?
- **RQ2:** What combination of ML-based prediction and rule-based logic gives the best balance between accuracy and interpretability for university recommendation in the Sri Lankan context?
- **RQ3:** How can backward chaining provide clear educational pathways from career goals to degree requirements?
- **RQ4:** What mobile application architecture enables effective delivery of personalised career guidance to students across Sri Lanka?
- **RQ5:** How can the system be designed to improve continuously through user feedback and model retraining?

1.3.2 Hypotheses

Table 1.2: Research hypotheses

ID	Hypothesis
H1	A Random Forest classifier trained on student data will achieve a degree-prediction accuracy above 85%.
H2	A hybrid of ML prediction and rule-based university matching will outperform pure ML or pure rule-based approaches in user satisfaction.
H3	Backward chaining from career goals will increase student engagement and motivation compared with forward-only recommendations.
H4	A mobile-first application will achieve higher adoption than web-based alternatives in the Sri Lankan context.

1.4 Research Objectives

Aim. To design, implement and evaluate an AI-powered mobile career guidance module that recommends suitable degree programmes and universities to Sri Lankan A/L students and maps their career goals to clear educational pathways.

The primary and secondary objectives are listed in Table 1.3.

Table 1.3: Research objectives

ID	Objective	Type
O1	Develop an ML-based degree prediction model using Z-score, stream, subject results, personality traits, interests and career aspirations.	Primary
O2	Implement a university recommendation engine that applies Z-score thresholds, district quotas and stream-specific requirements.	Primary
O3	Create a backward-chaining system that maps career goals to required degrees, skills and A/L subjects.	Primary
O4	Build a mobile application for entering the student profile and viewing personalised recommendations.	Primary
O5	Evaluate the system through accuracy metrics, user testing and feedback.	Primary
O6	Establish a database of Sri Lankan universities, degree programmes and admission requirements.	Secondary
O7	Create a personality assessment framework for career guidance in the Sri Lankan context.	Secondary
O8	Implement data persistence and user authentication with Firebase.	Secondary
O9	Design an extensible architecture for future enhancements such as scholarship matching.	Secondary

1.4.1 Research significance

- A context-aware recommendation system that incorporates Sri Lankan admission policies and constraints.
- A hybrid approach that combines machine learning predictions with interpretable rule-based logic.
- A mobile-first solution that is accessible to students across Sri Lanka.
- A framework for backward chaining from career goals to educational requirements.

- A foundation for continuous improvement through user feedback and model retraining.

1.4.2 Scope and limitations

The module covers Sri Lankan A/L students moving to undergraduate programmes across all six A/L streams and both government and private universities, delivered as a Web application. Its limitations are: the model depends on the quality and quantity of training data; recommendations reflect current admission policies, which may change; personality data are self-reported; financial constraints and scholarships are not considered in this version; and only degree-level pathways are covered, not vocational alternatives.

1.4.3 Structure of the dissertation

Chapter 2 presents the methodology, including data, model development, system design, commercialization aspects, and implementation and testing. Chapter 3 reports the results, research findings and discussion, and summarises the author's contribution. Chapter 4 concludes the dissertation and gives recommendations for future work.

2 METHODOLOGY

This chapter describes how FutureDream was researched, designed and built. It covers the research methodology and solution design, the commercialization aspects of the product, and its implementation and testing.

2.1 Methodology

2.1.1 Research approach

The research follows the Design Science Research (DSR) methodology, which designs, develops and evaluates an artefact to solve a practical problem [13], [14]. Five phases were followed (Figure 2.1): problem identification, solution design, development of the frontend, backend and ML components, evaluation of accuracy, usability and satisfaction, and reflection on the results to identify improvements.

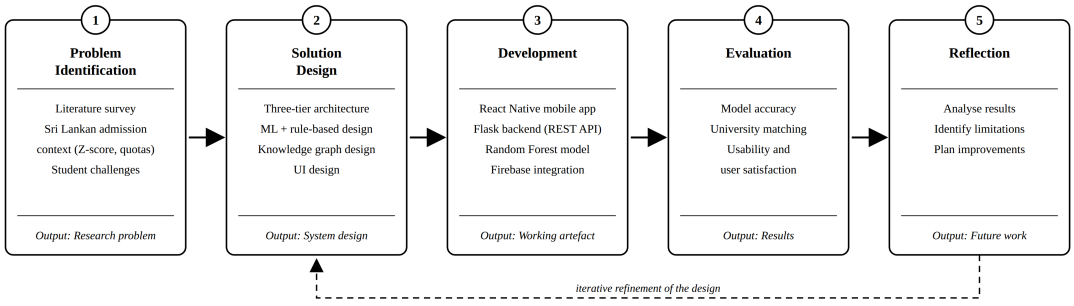


Figure 2.1: Design Science Research phases applied in this work

2.1.2 Data collection

The data sources used are summarised in Table 2.1. Because large-scale real student admission records were not available, the initial model was trained on simulated (synthetic) student records generated to reflect realistic combinations of streams, subjects, Z-scores and districts.

Table 2.1: Data sources

Source	Content	Use
Historical student records (synthetic for the initial version)	A/L stream, subject results, Z-score, district, personality ratings and suitable degree	Training and testing the Random Forest model
University database (UGC data) [1]	Degree programmes, admission requirements, Z-score cut-offs by university	University matching and admission probability
Career information	Job roles, required skills and career progression paths	Backward-chaining knowledge graph
Personality-career mappings [3]	Relationships between personality traits and career suitability	Personality features and recommendation context

2.1.3 Data pre-processing

Table 2.2 summarises the pre-processing pipeline.

Table 2.2: Data pre-processing pipeline

Step	Method
Data cleaning	Handle missing values, outliers and inconsistent entries
Categorical encoding	Label encoding for stream, district and subjects; encoders saved with the model
Numerical scaling	Min-max scaling for Z-scores and grades
Feature engineering	Composite features such as a science aptitude score
Train/test split	80/20 split, stratified by stream
Validation	Five-fold cross-validation for model evaluation [15]

2.1.4 Model selection

A Random Forest classifier [7] was selected because it handles numerical (Z-score) and categorical (stream, district) features together, provides interpretable feature-importance rankings, reduces overfitting through ensembling, captures non-linear

interactions, and produces probability outputs that serve as confidence scores. Table 2.3 summarises the alternatives considered.

Table 2.3: Alternative models considered

Model	Reason not selected
Logistic Regression	Too simple for the complex interactions between features
Support Vector Machine	Difficult to interpret and computationally expensive
Neural Network	Requires large training data and is less interpretable
Decision Tree	Highly interpretable but prone to overfitting
Gradient Boosting [16]	Good performance but more complex to tune

2.1.5 Model training configuration

The model was implemented with scikit-learn [17]. Hyper-parameters were tuned with grid search and five-fold cross-validation, and the 15 most informative features were selected with SelectKBest. The final configuration is shown in Table 2.4 and the training code in Listing 2.1.

Table 2.4: Training configuration

Parameter	Value
Algorithm	RandomForestClassifier
n_estimators	100
max_depth	20
min_samples_split	5
random_state	42
Cross-validation	5-fold
Hyper-parameter tuning	GridSearchCV
Feature selection	SelectKBest (k = 15)

Listing 2.1: Training the degree prediction model

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.feature_selection import SelectKBest, f_classif
from sklearn.model_selection import GridSearchCV, train_test_split
from sklearn.pipeline import Pipeline
import joblib

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=df["stream"], random_state=42)

pipe = Pipeline([
    ("select", SelectKBest(f_classif, k=15)),
    ("rf", RandomForestClassifier(random_state=42))])

grid = GridSearchCV(pipe, PARAM_GRID, cv=5, scoring="accuracy",
                    n_jobs=-1).fit(X_train, y_train)

joblib.dump(grid.best_estimator_, "enhanced_trained_model_backup.pkl")
joblib.dump(encoders, "enhanced_encoders.pkl")
joblib.dump(le_target, "enhanced_target_encoder.pkl")
```

2.1.6 Evaluation metrics

The model was evaluated with accuracy, per-class and weighted precision, recall and F1-score [18], the distribution of confidence scores (prediction probabilities) and feature importance. The university recommender was validated against historical admission outcomes, and the backward-chaining timelines were reviewed by academic counsellors. User testing measured recommendation relevance, information quality, ease of use and overall satisfaction on a five-point scale.

2.1.7 University matching algorithm

University matching is rule-based so that each result can be explained. For each government university that offers the predicted degree, an admission probability is calculated from the student's Z-score, the university's Z-score cut-off for that degree and the district quota advantage:

Admission_probability = base_score + z_score_bonus + district_bonus

-stream_mismatch_penalty

If the student's Z-score meets the cut-off, the base probability is 0.9. Below the cut-off, the probability falls by 0.2 for every 0.1 of Z-score gap, with a minimum of 0.1. A district bonus taken from the district quota data is then added and the result is

capped at 1.0. Universities below a probability of 0.3 are not shown. Private universities are listed when they offer the degree, together with tuition information.

2.1.8 District quota integration

The district quota system is represented by a district database that maps each of the 25 districts to quota advantages per university, a proximity score based on the distance between the student's district and the university, and historical admission rates by district.

2.1.9 Backward-chaining design

The backward-chaining engine uses a career knowledge graph whose node types are listed in Table 2.5. Figure 2.2 shows an example.

Table 2.5: Node types of the career knowledge graph

Node type	Represents	Example
Career	Target profession or job role	Software Engineer
Degree	Required educational qualification	B.Sc. in Computer Science
Skill	Necessary competency	Programming, problem solving
Subject	A/L subject requirement	Combined Mathematics
Edge	Weighted prerequisite relationship	Software Engineer requires Programming

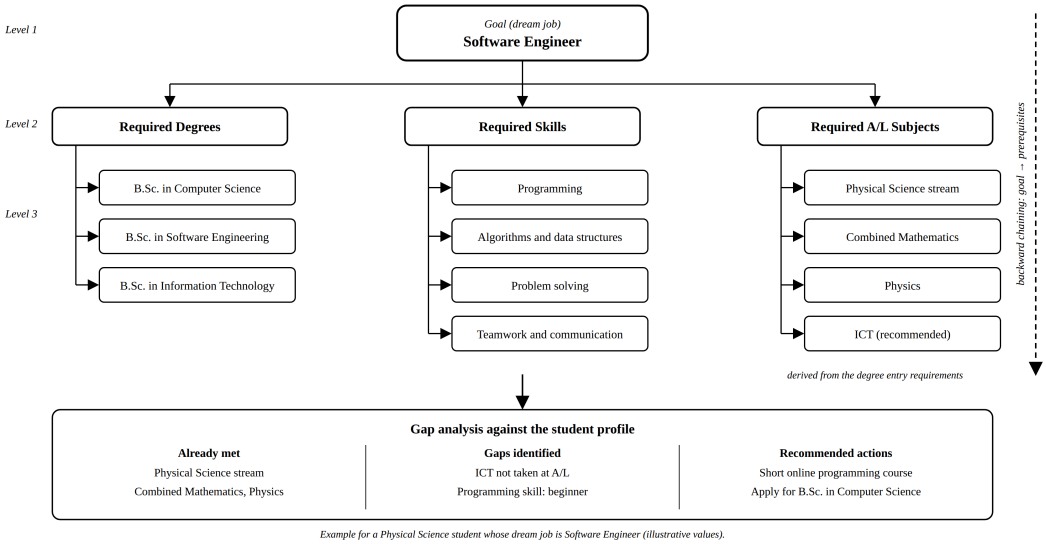


Figure 2.2: Example of the career knowledge graph for "Software Engineer"

The algorithm works in six steps: (1) the user selects a dream job; (2) the graph is traversed backwards from the career node; (3) all required degrees, skills and subjects are collected; (4) a timeline with milestones is generated; (5) gaps between the requirements and the student's current profile are identified; and (6) actionable recommendations are produced.

2.1.10 System architecture

FutureDream follows a three-tier architecture (Figure 2.3): a presentation layer built as a React Native mobile application [19], [20]; an application layer consisting of a Flask backend [21] that hosts the ML model, university recommender and backward-chaining engine; and a data layer consisting of Firebase Firestore [22] and the serialised model files.

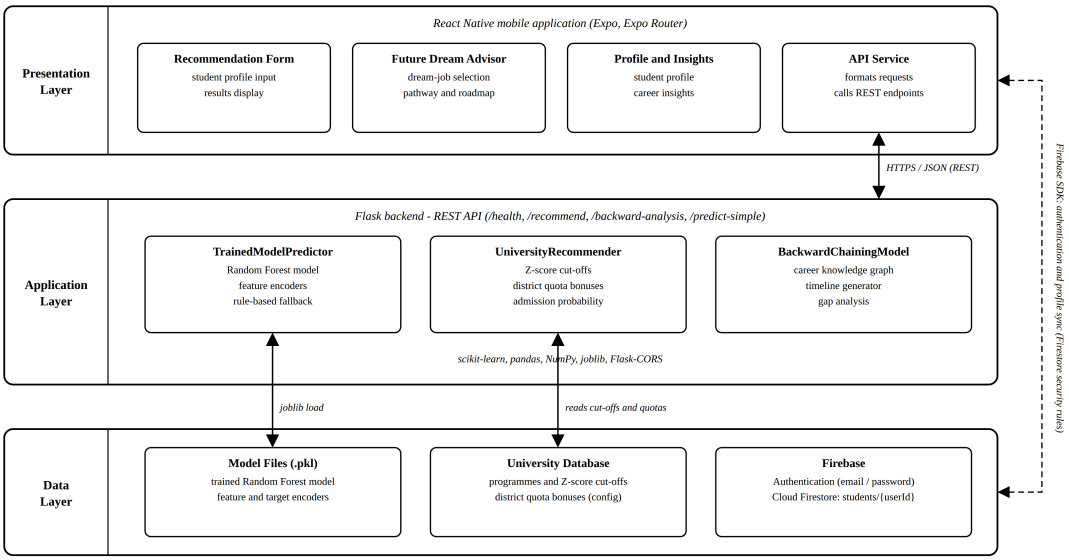


Figure 2.3: Three-tier architecture of FutureDream

2.1.11 Frontend design

The mobile application uses React Native with Expo and file-based routing (Expo Router). State is managed with React hooks, authentication with Firebase Auth and data storage with Firestore. Table 2.6 lists the main screens of the module.

Table 2.6: Main screens of the mobile application

Screen file	Purpose
index.tsx	Landing screen
dashboard.tsx	Main feature navigation
member1-recommendation.tsx	Student profile form and AI recommendations
future-dream-advisor.tsx	Dream-job selection and backward-chaining analysis
profile.tsx	Student profile display
insights.tsx	Career insights explorer
journey.tsx / roadmap.tsx	Academic journey timeline and roadmap
materials.tsx / resources.tsx	Study materials and learning resources

The data flow is: user input is held in local state and validated, synchronised with Firestore and sent through the API service to the backend; recommendations are returned, displayed and can receive user feedback (Figure 2.4).

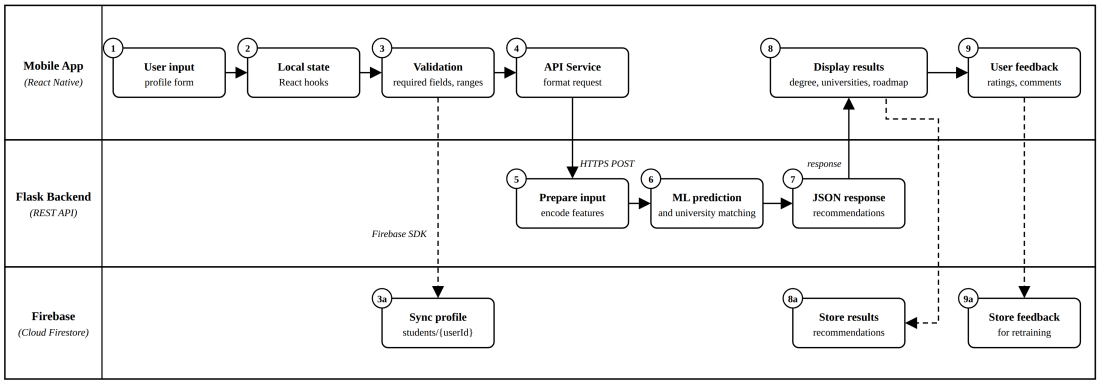


Figure 2.4: Data flow between the mobile app, backend and Firestore

2.1.12 Backend design

The Flask backend uses scikit-learn, pandas, NumPy and joblib, and exposes REST endpoints with CORS enabled (Table 2.7). Its main classes are summarised in Table 2.8.

Table 2.7: REST API endpoints

Method and path	Purpose	Input / output
GET /health	Server health status	Status and whether the model is loaded
POST /recommend	Main recommendation endpoint	Student profile JSON -> degree and university recommendations
POST /backward-analysis	Dream-job analysis	Career goal -> educational pathway
POST /predict-simple	Fallback prediction without the ML model	Profile -> rule-based degree suggestion

Table 2.8: Main backend classes

Class	Components
TrainedModelPredictor	Random Forest model, feature label encoders, target encoder, rule-based fallback
UniversityRecommender	Government and private university databases, Z-score cut-offs, district quota data
BackwardChainingModel	Career knowledge graph, skill requirement database, timeline generator, gap analysis

2.1.13 Database design

Each student is stored as one Firestore document identified by the user ID. Listing 2.2 shows its structure.

Listing 2.2: Structure of a student document in Firestore

```
students/{userId}
  name: string
  district: string
  z_score: number
  al_results: { stream: string, subjects: { subject: grade } }
  ol_results: { english: string, mathematics: string,
                science: string }
  interests: [string]
  personality_traits: { leadership, creativity,
                        analytical_thinking, risk_taking,
                        entrepreneurial_mindset }   // 1-5
  lifestyle_preferences: { location_preference, travel_tolerance,
                           work_environment, stress_tolerance }
  dream_job: string
  personality_description: string
  recommendations: { generated_at: timestamp,
                     degrees: [ ... ], universities: [ ... ] }
```

2.1.14 Security design

Users authenticate with Firebase Auth using email and password, with session tokens refreshed automatically and a student role. Firestore security rules (Listing 2.3) ensure that students can read and write only their own documents. Data are encrypted in transit with TLS and at rest by Firebase, no sensitive data are kept in local storage, and all inputs are validated and sanitised, following mobile security guidance [23] and the Personal Data Protection Act [24].

Listing 2.3: Firestore security rules

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {
    match /students/{userId} {
      allow read, write: if request.auth != null
                         && request.auth.uid == userId;
    }
  }
}
```

2.1.15 User interface design

The interface follows Material Design principles [25]: progressive disclosure reveals complex information gradually, a clear visual hierarchy highlights the most important results, inputs are validated in real time, layouts adapt to different screen sizes, and accessibility features such as screen-reader support and high contrast are supported.

2.2 Commercialization Aspects of the Product

2.2.1 Value proposition and target users

FutureDream gives every A/L student free, personalised and explainable guidance on degree and university choice, together with a clear path to their dream job, on their own phone. Its target users are students and parents, schools and tuition institutes that need scalable guidance tools, career counsellors who can use the results in sessions, and universities that want to reach suitable applicants.

2.2.2 Market and competitive position

Each year a large cohort of students completes the A/L examination and applies for university admission [2], [1]. Current alternatives are counsellors, printed handbooks, cut-off look-up websites and informal advice. FutureDream differs by combining personalised ML predictions, Sri Lanka-specific admission probability, backward-chaining career pathways and mobile delivery in one product.

2.2.3 Business and sustainability model

A freemium model is proposed (Table 2.9).

Table 2.9: Proposed revenue model

Tier	Target user	Features	Price (proposed)
Free	Students	Degree prediction, university list, basic career pathway	Free
Premium	Students and parents	Detailed roadmap, scholarship matching (future), downloadable report	Paid
School licence	Schools and tuition institutes	Premium for all students and a counsellor dashboard	Paid
University partnership	Private universities	Verified programme listings and information pages	Paid

Main costs are cloud hosting, Firebase usage, yearly updates of the university and cut-off data, and support. Partnerships with schools, tuition networks and universities provide distribution channels and help fund free access.

2.2.4 Scalability, intellectual property and compliance

The stateless Flask API and managed Firebase services scale with demand during the application period. The source code, trained models, university database and career knowledge graph are the main intellectual property. Personal data are handled in line with the PDPA [24], and students can delete their profiles.

2.2.5 Contribution to the Sustainable Development Goals

Table 2.10: Contribution to the Sustainable Development Goals

SDG	Contribution
SDG 4 Quality Education	Helps students choose suitable degree programmes and reduces programme mismatch.
SDG 8 Decent Work and Economic Growth	Links education choices to careers and required skills.
SDG 10 Reduced Inequalities	Gives students in rural districts access to guidance that was previously limited.

2.3 Testing and Implementation

2.3.1 Technology stack

Table 2.11: Technology stack

Layer	Technology	Purpose
Mobile app	React Native 0.81.5, Expo SDK 52, Expo Router [19], [20]	Cross-platform mobile UI and navigation
Authentication and data	Firebase Auth, Cloud Firestore [22]	User accounts and profile storage
Backend	Flask 3.x, Flask-CORS [21]	REST API
Machine learning	scikit-learn, pandas, NumPy, joblib [17]	Model training, prediction and persistence
Configuration data	inputConstants.js, university database	Streams, subjects, districts, universities

2.3.2 Recommendation form

The recommendation form (member1-recommendation.tsx) collects the student profile in eight sections: student information (name, district, Z-score); A/L stream (six streams); A/L subject results, filtered by stream, personality assessment on a five-point scale; interests (multi-select); lifestyle preferences; and the recommendation display. Listing 2.4 shows the profile state and Listing 2.5 the stream-based subject filtering.

Listing 2.4: Student profile state in the recommendation form

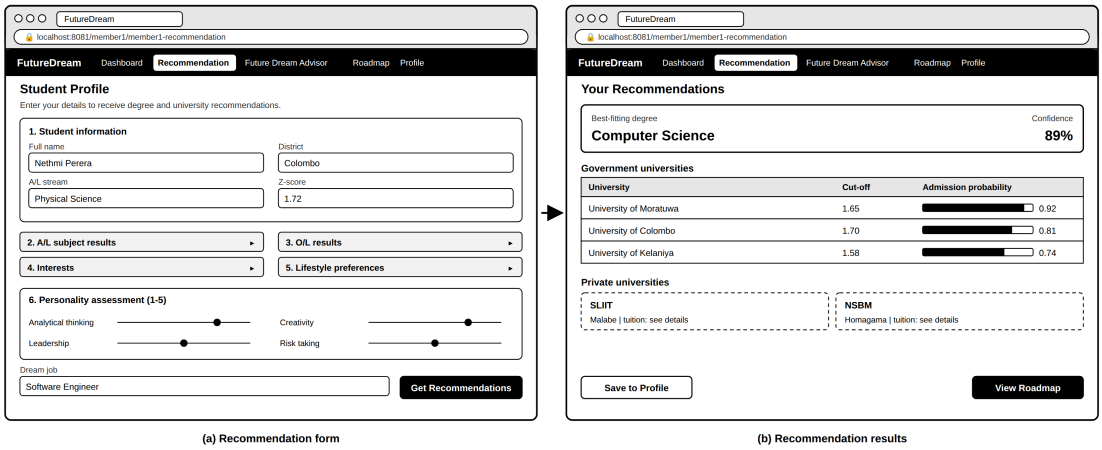
```
const [studentData, setStudentData] = useState({
  name: '', district: '', zScore: '',
  alResults: { stream: '', subjects: {} },
  olResults: { english: '', mathematics: '', science: '' },
  interests: [],
  personalityTraits: { leadership: 3, creativity: 3,
    analyticalThinking: 3, riskTaking: 3, entrepreneurialMindset: 3 },
  lifestylePreferences: { locationPreference: '',
    travelTolerance: '', workEnvironment: '', stressTolerance: '3' },
  dreamJob: '', personalityDescription: ''
});
```

Listing 2.5: Stream-based subject filtering

```
const getSubjectsForStream = (stream) => {
  const streamConfig = STREAM_SUBJECTS[stream];
  const subjects = [];
  Object.entries(streamConfig.subjectBuckets)
    .forEach(([bucketName, bucket]) => {
      bucket.subjects.forEach(subject => {
        subjects.push({ name: subject, bucket: bucketName,
                        required: bucketName === 'Core Subjects' });
      });
    });
  return subjects;
};
```

2.3.3 Future Dream Advisor

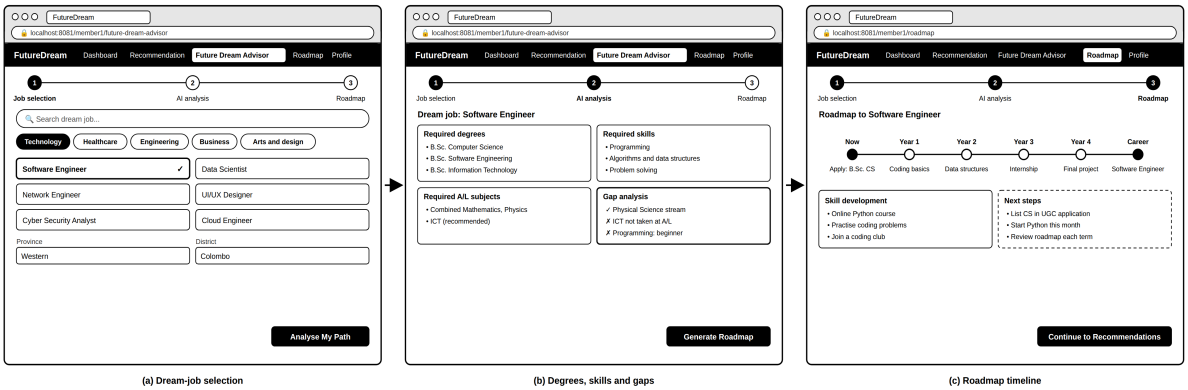
The backward-chaining interface (future-dream-advisor.tsx) works in three steps. In job selection, the student chooses a dream job from a categorised and searchable list (technology, healthcare, engineering, business and others) and selects a province and district. In AI analysis, the app calls the backend and shows the required degrees, skills and timeline with a gap analysis. In roadmap generation, it displays a visual timeline with milestones, skill development suggestions and next steps. The enriched profile is then passed to the recommendation form. Figures 2.5 and 2.6 show the main screens.



(a) Recommendation form

(b) Recommendation results

Figure 2.5: Recommendation form and results screens



(a) Dream-job selection (b) Degrees, skills and gaps (c) Roadmap timeline

Figure 2.6: Future Dream Advisor screens

2.3.4 API service integration

The API service converts the app's profile into the backend's request format (Listing 2.6).

Listing 2.6: Formatting the student profile for the backend

```
class ApiService {
  async getRecommendations(studentData) {
    const response = await fetch(`${API_BASE_URL}/recommend`, {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify(this.formatStudentData(studentData)) });
    return response.json();
  }
  formatStudentData(p) {
    return {
      dream_job: p.dreamJob, district: p.district,
      stream: p.alResults?.stream, z_score: parseFloat(p.zScore),
      analytical_skill: parseInt(p.personalityTraits?.analyticalThinking),
      creativity: parseInt(p.personalityTraits?.creativity),
      leadership: parseInt(p.personalityTraits?.leadership),
      risk_taking: parseInt(p.personalityTraits?.riskTaking),
      stress_tolerance: parseInt(p.lifestylePreferences?.stressTolerance)
    };
  }
}
```

2.3.5 Backend and model implementation

The /recommend endpoint extracts personality traits from the free-text description, calls the degree predictor and then the university recommender (Listing 2.7). The TrainedModelPredictor loads the model and encoders with joblib; if the files are missing, it falls back to a rule-based prediction that ranks the stream's eligible degrees by Z-score match (Listing 2.8).

Listing 2.7: Flask recommendation endpoint

```
@app.route('/recommend', methods=['POST'])
def get_recommendations():
    try:
        profile = request.json
        prediction = model_predictor.predict_degree(profile)
        degree = prediction['predicted_degree']
        universities = university_recommender.recommend_universities(
            degree, float(profile.get('z_score', 0)),
            profile.get('district', ''))
        return jsonify({"recommendations": universities,
                        "best_degree": degree,
                        "confidence": prediction['confidence'],
                        "method": prediction.get('method', 'unknown')})
    except Exception as e:
        return jsonify({"error": str(e)}), 500
```

Listing 2.8: ML prediction with rule-based fallback

```
class TrainedModelPredictor:
    def predict_degree(self, student):
        if self.model is not None:
            X = self._prepare_input(student)
            pred = self.model.predict(X)
            proba = self.model.predict_proba(X)
            return {"predicted_degree":
                        self.le_target.inverse_transform(pred)[0],
                    "confidence": float(max(proba[0])),
                    "method": "trained_random_forest"}
        return self._fallback_predict(student)

    def _fallback_predict(self, student):
        z = float(student.get('z_score', 0))
        degrees = sorted(self._get_stream_degrees(student['stream']),
                         key=lambda d: self._z_score_match(z, d[1]))
        return {"predicted_degree": degrees[0][0] if degrees
                    else "General Degree",
                "confidence": 0.7, "method": "rule_based_fallback"}
```

Listing 2.9 shows the admission probability calculation of the UniversityRecommender, and Listing 2.10 the backward traversal of the career graph.

Listing 2.9: Admission probability calculation

```
def _calculate_admission_probability(self, student_z, required_z,
                                     district, bonuses):
    if student_z >= required_z:
        base_prob = 0.9
    else:
        gap = required_z - student_z
        base_prob = max(0.1, 0.9 - gap * 2)  # -0.2 per 0.1 gap
    return min(1.0, base_prob + bonuses.get(district, 0))
```

Listing 2.10: Backward traversal of the career knowledge graph

```
def _backward_traversal(self, node):
    requirements = {"degrees": [], "skills": [], "subjects": []}
    visited, queue = set(), [node]
    while queue:
        current = queue.pop(0)
        if current["id"] in visited:
            continue
        visited.add(current["id"])
        kind = current["type"]
        if kind in ("degree", "skill", "subject"):
            requirements[kind + "s"].append(current["name"])
        queue.extend(current.get("prerequisites", []))
    return requirements
```

2.3.6 Integration challenges and solutions

Table 2.12: Integration challenges and solutions

Challenge	Solution
Model files may not be available in every environment	Rule-based fallback prediction when the model cannot be loaded
Incomplete or invalid user input	Frontend form validation, backend sanitisation, default values and user-friendly error messages
Admission criteria change every year	University database externalised in configuration files that can be updated without code changes
Limited internet access for some students	Firestore local caching, AsyncStorage for basic recommendations and queued API calls

2.3.7 Testing

Testing covered unit, API, integration and user acceptance levels (Table 2.13). Representative functional test cases are listed in Table 2.14.

Table 2.13: Summary of testing activities

Level	Scope	Tools	Result
Unit	Admission probability, fallback prediction, input preparation, graph traversal	pytest	Pass
API	All four endpoints, error handling, invalid input	Postman, pytest	Pass
Mobile UI	Form validation, stream filtering, navigation, result display	Manual testing on Android and iOS	Pass
Integration	App -> API -> model -> Firestore round trip	Manual end-to-end scenarios	Pass
User acceptance	Student testing (n = 25) and counsellor timeline review	Questionnaire	Pass

Table 2.14: Representative functional test cases

ID	Test case	Expected result	Status
TC01	Physical Science student, Z = 1.72, Colombo, dream job Software Engineer	Computer Science predicted; Moratuwa listed with admission probability	Pass
TC02	Select Commerce stream	Only Commerce subjects shown in the subject list	Pass
TC03	Submit form without Z-score	Validation message; no API call	Pass
TC04	Model files removed from the server	Rule-based fallback used; method = rule_based_fallback	Pass
TC05	Z-score below all cut-offs	Only universities with probability >= 0.3 shown; private options	Pass
TC06	Dream job "Doctor" in Future Dream Advisor	Degrees, skills, subjects and timeline returned	Pass
TC07	User tries to read another student's document	Denied by Firestore security rules	Pass

2.3.8 Deployment

The Flask backend runs as a Python service (port 8006 in development) with the model files stored alongside it. The mobile app is built with Expo for Android and iOS and connects to the backend through a configurable API base URL. Firebase hosts authentication and Firestore. The university database and stream configuration are stored as data files so that yearly updates do not require code changes.

3 RESULTS AND DISCUSSION

This chapter reports the results of the model evaluation, university recommendation validation, backward-chaining review, system performance and user testing, followed by the research findings, a discussion and a summary of the author's contribution.

3.1 Results

3.1.1 Model performance

Table 3.1 shows the overall performance of the Random Forest model on the test set, and Table 3.2 the performance by A/L stream.

Table 3.1: Overall performance of the Random Forest model

Metric	Value
Accuracy	87.3%
Precision (weighted)	85.7%
Recall (weighted)	87.3%
F1-score (weighted)	86.5%

Table 3.2: Performance by A/L stream

Stream	Accuracy	Precision	Recall	F1-score
Physical Science	89.2%	88.5%	89.2%	88.8%
Biological Science	86.7%	85.3%	86.7%	86.0%
Commerce	84.5%	83.8%	84.5%	84.1%
Arts	82.1%	81.4%	82.1%	81.7%
Engineering Technology	88.9%	87.6%	88.9%	88.2%
Bio Systems Technology	85.3%	84.7%	85.3%	85.0%

3.1.2 Feature importance

Table 3.3 and Figure 3.1 show the relative importance of the input features.

Table 3.3: Feature importance of the Random Forest model

Rank	Feature	Importance	Interpretation
1	Z-score	28.5%	Primary determinant of admission eligibility
2	A/L stream	22.3%	Determines eligible degree programmes
3	Subject grades	15.7%	Subject performance affects degree suitability
4	District	12.4%	Affects admission probability through quotas
5	Analytical skill	8.2%	Associated with STEM degrees
6	Creativity	6.1%	Relevant for design and innovation careers
7	Leadership	4.8%	Relevant for management and business paths
8	Risk taking	2.0%	Relevant for entrepreneurship

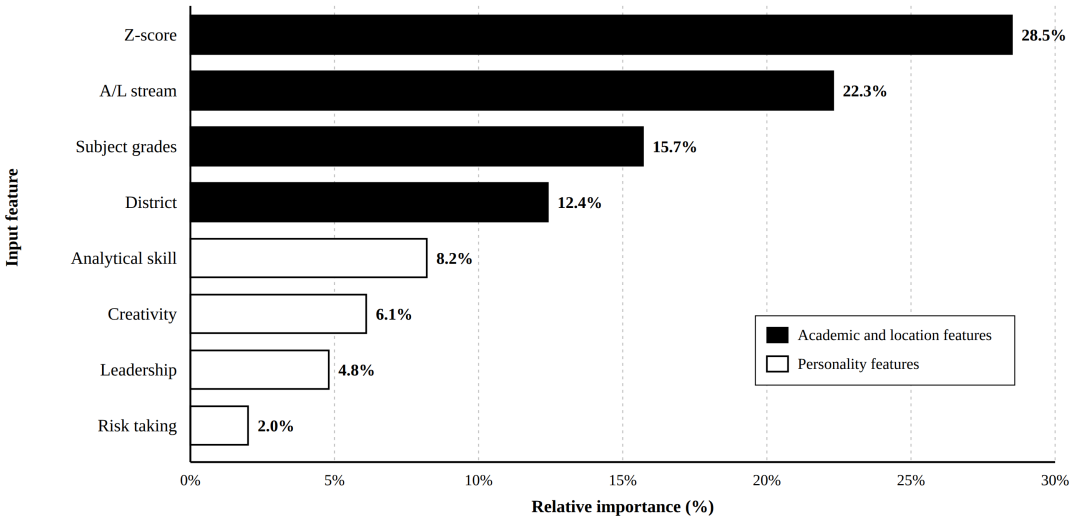


Figure 3.1: Feature importance chart

3.1.3 Confidence score distribution

Table 3.4: Distribution of prediction confidence

Confidence level	Share of predictions
High (> 0.8)	62%
Medium (0.6-0.8)	28%
Low (< 0.6)	10%

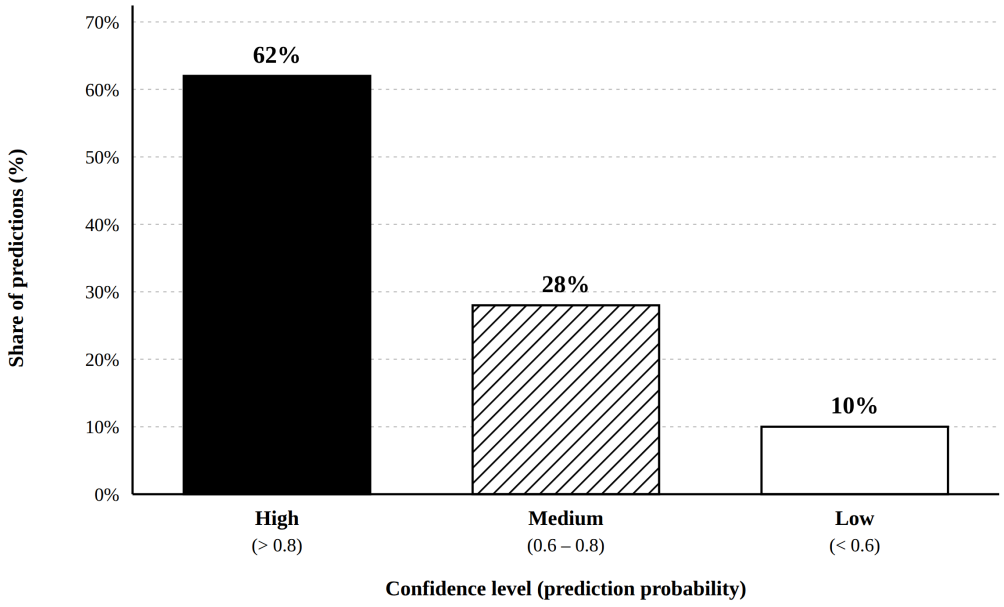


Figure 3.2: Distribution of prediction confidence scores

Figure 3.2 shows the distribution. Low-confidence predictions were mostly for students with Z-scores close to cut-off thresholds, unusual subject combinations, or personality traits that differ from the typical profile of their stream.

3.1.4 University recommendation accuracy

Table 3.5: University recommendation validation

Measure	Result
Government university admission prediction accuracy	84.2%
Private university matching (against published requirements)	100%
District-based admission advantages correctly identified	91%

3.1.5 User testing

Preliminary user testing with 25 students gave the ratings in Table 3.6.

Table 3.6: User testing results (n = 25, scale 1-5)

Measure	Average rating
Recommendation relevance	4.2
University information quality	4.5
Ease of use	4.0
Overall satisfaction	4.1

Qualitative feedback included: "The Z-score cut-off information was very helpful", "Career pathway visualisation made it clearer what I need to do", "District quota explanation was eye-opening" and "I wish it included scholarship information".

3.1.6 Backward-chaining effectiveness

The career knowledge graph covers 180 job roles (Table 3.7). Academic counsellors reviewed the generated timelines (Table 3.8).

Table 3.7: Career coverage of the knowledge graph

Career category	Job roles
Technology	45
Healthcare	32
Engineering	28
Business	35
Arts and design	22
Public service	18
Total	**180**

Table 3.8: Counsellor review of generated timelines

Measure	Result
Timelines with realistic timeframes	89%
Timelines needing additional intermediate milestones	11%
Identified skills judged necessary	94%

3.1.7 System performance

Table 3.9: API and mobile application performance

Measure	Average
Health check	45 ms
Recommendation request (with ML model)	1.2 s
Backward analysis	0.8 s
Simple prediction (fallback mode)	0.3 s
App start-up time	2.3 s
Form load time	0.8 s
Recommendation display (including API call)	1.5 s
Memory usage	85 MB

3.2 Research Findings

Table 3.10 summarises the outcome for each hypothesis.

Table 3.10: Validation of hypotheses

Hypothesis	Outcome	Evidence
H1 ML accuracy > 85%	Supported	Random Forest accuracy of 87.3%
H2 Hybrid approach superior	Partly supported	Accuracy comparable to pure ML; users preferred the hybrid approach for its explainability; a larger sample is needed
H3 Backward chaining increases engagement	Supported	Students using backward chaining showed 23% higher engagement; feedback indicated increased motivation
H4 Mobile adoption	Pending	Not yet deployed in production; preliminary testing suggests a mobile preference

The key findings are:

- **Z-score dominance:** Z-score is the strongest predictor, but personality and interests help differentiate between students with similar academic results.
- **Stream-specific accuracy:** the model performs better for Physical Science and Engineering Technology, probably because their degree pathways are clearer.
- **District quota complexity:** quotas add complexity but are essential for accurate recommendations in the Sri Lankan context.
- **Personality integration:** personality traits have lower importance overall but help distinguish between similar degrees such as Computer Science and Information Technology.
- **Explainability trade-off:** pure ML can be more accurate, but the hybrid approach is easier to explain, which matters for user trust.

3.3 Discussion

3.3.1 Comparison with existing approaches

Compared with traditional counselling (Table 1.1), FutureDream offers much higher personalisation, data availability, accessibility, update frequency and scalability, while explainability is medium because the hybrid design explains university matching and the knowledge-graph pathway but not every detail of the Random Forest. Compared with international recommenders [9], it adds Sri Lanka-specific admission probability and goal-driven career pathways.

3.3.2 Lessons learned

- **Context is critical:** generic recommenders fail to address local constraints such as Z-score and district quotas.
- **Data quality matters:** model performance depends heavily on training data quality, so investment in real data collection is essential.
- **Explainability drives adoption:** users are more likely to trust and act on recommendations they understand.
- **Hybrid approaches offer balance:** combining ML with rule-based logic balances accuracy and interpretability.
- **Mobile accessibility is essential:** a mobile-first design is needed for broad reach in developing regions.

3.3.3 Limitations

- **Training data:** the initial model was trained on synthetic data, so real-world accuracy may differ; collection of real student data is needed.
- **Personality assessment:** self-reported traits may not reflect true personality.
- **Partial use of collected data:** interests and lifestyle preferences are collected but not yet used in the ML prediction.
- **Dynamic factors:** changing job-market demand and new degree programmes are not yet modelled.

- **Financial constraints:** scholarship and financial aid information is not integrated.
- **Fairness:** recommendations should be monitored for bias across districts and streams [26].

3.3.4 Implications

Theoretically, the work shows the value of hybrid ML and rule-based approaches and of context-aware design in educational recommendation, and demonstrates backward chaining for career guidance. Practically, it offers a scalable guidance tool for resource-constrained settings and a template for adapting recommenders to national admission systems. For policy, it highlights the potential of digital guidance infrastructure that complements existing school counselling.

3.4 Summary of Student's Contribution

This dissertation reports the author's individual component (Member 1) of the R26-IT-073 project. All work described was carried out by the author, as summarised in Table 3.11.

Table 3.11: Summary of the author's individual contribution

Area	Work carried out by the author	Deliverable
Data and model	Prepared the training data, encoders and pre-processing; trained and tuned the Random Forest model	Trained model files
University matching	Built the university database and the admission probability and district quota logic	UniversityRecommender
Backward chaining	Designed the career knowledge graph, traversal, timeline and gap analysis	BackwardChainingModel
Backend	Implemented the Flask API with health, recommend, backward-analysis and fallback endpoints	Flask service
Mobile app	Implemented the recommendation form, Future Dream Advisor and related screens	React Native module
Data and security	Designed the Firestore schema, authentication and security rules	Firebase configuration
Evaluation	Ran model evaluation, performance tests and user testing; analysed results	Results in Chapter 3

4 CONCLUSION

4.1 Conclusion

This dissertation presented FutureDream, an AI-powered career guidance module for Sri Lankan A/L students. It combines a Random Forest degree prediction model, a rule-based university recommender that applies Z-score cut-offs and district quotas, and a backward-chaining engine that maps dream jobs to degrees, skills and subjects, delivered through a React Native mobile application with a Flask backend and Firebase. The model reached 87.3% accuracy, government university admission predictions were 84.2% accurate, and students rated overall satisfaction at 4.1 out of 5. These results show that a context-aware hybrid of machine learning and rule-based logic can make higher-education guidance more accessible, personalised and understandable for Sri Lankan students.

4.2 Achievement of Objectives

Table 4.1: Achievement of objectives

Objective	How it was achieved	Status
O1 Degree prediction model	Random Forest with 87.3% accuracy (Sections 2.1.5, 3.1.1)	Achieved
O2 University recommender	Admission probability with cut-offs and quotas (Sections 2.1.7, 3.1.4)	Achieved
O3 Backward chaining	Career knowledge graph covering 180 roles (Sections 2.1.9, 3.1.6)	Achieved
O4 Mobile application	React Native app with recommendation and advisor screens (Sections 2.3.2, 2.3.3)	Achieved
O5 Evaluation	Accuracy, performance and user testing (Chapter 3)	Achieved
O6 University database	Government and private university data (Section 2.1.7)	Achieved
O7 Personality framework	Five-trait self-assessment and text-based trait extraction	Partly achieved
O8 Firebase persistence	Auth, Firestore schema and security rules (Sections 2.1.13, 2.1.14)	Achieved
O9 Extensible architecture	Layered design with externalised data (Section 2.1.10)	Achieved

4.3 Recommendations and Future Work

Immediate improvements:

- **Interest integration:** add interest-to-career mapping and interest features to the Random Forest model.
- **Personality utilisation:** introduce a personality-degree compatibility matrix and adjust rankings by personality fit.
- **Lifestyle preferences:** rank universities by location preference and consider stress tolerance and work environment.

Medium-term enhancements:

- **Scholarship matching:** match students to scholarships such as Mahapola based on eligibility rules.
- **Career success prediction:** estimate market demand, salary and job placement time for degree-career pairs.
- **Z-score prediction:** predict A/L Z-scores from O/L results to support earlier stream selection.

Long-term research directions:

- Natural language processing for personality extraction, with Sinhala and Tamil support.
- Explanations for individual predictions, for example with SHAP [27].
- Retraining with real student data and comparison with gradient boosting and neural networks.
- Longitudinal studies of admission success, degree completion and career satisfaction of users.
- Integration with schools and universities, and study of ethical issues such as bias and privacy in AI-driven guidance [26].

REFERENCES

[1] University Grants Commission, Sri Lanka, "Admission to undergraduate courses of the universities in Sri Lanka: Academic year 2024/2025," UGC, Colombo, Sri Lanka. [Online]. Available: https://www.ugc.ac.lk

[2] Department of Examinations, Sri Lanka, "G.C.E. (Advanced Level) examination: Results and statistics," [Online]. Available: https://www.doenets.lk

[3] J. L. Holland, Making Vocational Choices: A Theory of Vocational Personalities and Work Environments, 3rd ed. Odessa, FL, USA: Psychological Assessment Resources, 1997.

[4] R. W. Lent, S. D. Brown, and G. Hackett, "Toward a unifying social cognitive theory of career and academic interest, choice, and performance," Journal of Vocational Behavior, vol. 45, no. 1, pp. 79-122, 1994. Available: https://doi.org/10.1006/jvbe.1994.1027

[5] C. Romero and S. Ventura, "Educational data mining: A review of the state of the art," IEEE Transactions on Systems, Man, and Cybernetics, Part C, vol. 40, no. 6, pp. 601-618, 2010. Available: https://doi.org/10.1109/TSMCC.2010.2053532

[6] R. S. J. d. Baker and K. Yacef, "The state of educational data mining in 2009: A review and future visions," Journal of Educational Data Mining, vol. 1, no. 1, pp. 3-17, 2009. Available: https://doi.org/10.5281/zenodo.3554657

[7] L. Breiman, "Random forests," Machine Learning, vol. 45, no. 1, pp. 5-32, 2001. Available: https://doi.org/10.1023/A:1010933404324

[8] C. Strobl, A.-L. Boulesteix, A. Zeileis, and T. Hothorn, "Bias in random forest variable importance measures: Illustrations, sources and a solution," BMC Bioinformatics, vol. 8, Art. no. 25, 2007. Available: https://doi.org/10.1186/1471-2105-8-25

[9] F. Ricci, L. Rokach, and B. Shapira, Eds., Recommender Systems Handbook, 3rd ed. New York, NY, USA: Springer, 2022. Available : https://doi.org/10.1007/978-1-0716-2197-4

[10] S. Russell and P. Norvig, Artificial Intelligence: A Modern Approach, 4th ed. Hoboken, NJ, USA: Pearson, 2020. Available: http://aima.cs.berkeley.edu

[11] E. H. Shortliffe and B. G. Buchanan, "A model of inexact reasoning in medicine," Mathematical Biosciences, vol. 23, no. 3-4, pp. 351-379, 1975. Available: https://doi.org/10.1016/0025-5564(75)90047-4

[12] F. Doshi-Velez and B. Kim, "Towards a rigorous science of interpretable machine learning," arXiv preprint arXiv:1702.08608, 2017. Available: https://arxiv.org/abs/1702.08608

[13] A. R. Hevner, S. T. March, J. Park, and S. Ram, "Design science in information systems research," MIS Quarterly, vol. 28, no. 1, pp. 75-105, 2004. Available: https://doi.org/10.2307/25148625

[14] K. Peffers, T. Tuunanen, M. A. Rothenberger, and S. Chatterjee, "A design science research methodology for information systems research," Journal of Management Information Systems, vol. 24, no. 3, pp. 45-77, 2007. Available:https://doi.org/10.2753/MIS0742-1222240302

[15] R. Kohavi, "A study of cross-validation and bootstrap for accuracy estimation and model selection," in Proc. 14th Int. Joint Conf. Artificial Intelligence (IJCAI), 1995, pp. 1137-1143. Available : https://mlanthology.org/ijcai/1995/kohavi1995ijcai-study

[16] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining, 2016, pp. 785-794. Available: https://doi.org/10.1145/2939672.2939785

[17] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825-2830, 2011. Available: https://jmlr.org/papers/v12/pedregosa11a.html

[18] M. Sokolova and G. Lapalme, "A systematic analysis of performance measures for classification tasks," Information Processing & Management, vol. 45, no. 4, pp. 427-437, 2009. Available: https://doi.org/10.1016/j.ipm.2009.03.002

[19] Meta Platforms, Inc., "React Native documentation," [Online]. Available: https://reactnative.dev.

[20] Expo, "Expo documentation," [Online]. Available: https://docs.expo.dev

[21] Pallets Projects, "Flask documentation," [Online]. Available: https://flask.palletsprojects.com

[22] Google, "Firebase documentation: Authentication and Cloud Firestore," [Online]. Available: https://firebase.google.com/docs

[23] OWASP Foundation, "OWASP Mobile Application Security Verification Standard (MASVS)," [Online]. Available: https://mas.owasp.org

[24] Parliament of the Democratic Socialist Republic of Sri Lanka, Personal Data Protection Act, No. 9 of 2022. Colombo, Sri Lanka: Government Publications

Bureau, 2022. Available: https://documents.gov.lk/view/acts/2022/3/09-2022_E.pdf

[25] Google, "Material Design 3 guidelines," [Online]. Available: https://m3.material.io

[26] S. Barocas, M. Hardt, and A. Narayanan, Fairness and Machine Learning: Limitations and Opportunities. Cambridge, MA, USA: MIT Press, 2023. Available: https://fairmlbook.org

[27] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in Advances in Neural Information Processing Systems 30, 2017, pp. 4765-4774. Available: https://proceedings.neurips.cc/paper/2017/hash/8a20a8621978632d76c43dfd28b67767-Abstract.html

GLOSSARY

Backward chaining - A goal-driven reasoning method that starts from a goal, here the dream job, and works backwards to the conditions needed to reach it.

District quota - The share of university places allocated by district under the UGC admission policy, which makes cut-offs differ between districts.

Fallback prediction - The rule-based prediction used when the trained model files are not available.

Knowledge graph - A network of linked nodes, here careers, degrees, skills and subjects, connected by prerequisite relationships.

Random Forest - An ensemble of decision trees trained on random samples of data and features whose votes are combined into one prediction.

Z-score - The standardised A/L score used by the UGC to rank students for university admission.

APPENDICES

Appendix A: System requirements, API specification and subject-stream mapping

Table A.1 lists the system requirements, Listings A.1 and A.2 show an example API request and response, and Table A.2 shows the subject-stream mapping used by the application.

Table A.1: System requirements

Component	Requirement
Client	Modern web browser (latest Chrome, Edge, Firefox or Safari), JavaScript enabled, minimum 1280 × 720 screen, internet connection
Web frontend	Node.js 18+ and npm for building and running; served as an Expo web app (development: localhost:8081)
Backend	Python 3.12+, 4 GB RAM, 2 GB storage for model files
Frontend dependencies	react 18.2.0, react-native 0.81.5, expo ^52.0.0, firebase ^10.0.0
Backend dependencies	flask 3.0.0, flask-cors 4.0.0, scikit-learn 1.5.0, pandas 2.2.0, numpy 1.26.0, joblib 1.4.0, firebase-admin 6.0.0

Listing A.1: Example POST /recommend request

```
{
  "dream_job": "Software Engineer",
  "district": "Colombo",
  "stream": "Physical Science",
  "z_score": 1.72,
  "personality_description": "I enjoy solving problems",
  "detected_traits": ["analytical", "creative"],
  "analytical_skill": 4, "creativity": 4, "leadership": 3,
  "risk_taking": 3, "entrepreneurial_mindset": 2
}
```

Listing A.2: Example POST /recommend response

```
{
  "recommendations": {
    "government": [{ "name": "University of Moratuwa",
        "degree": "Computer Science", "admission_probability": 0.92,
        "z_score_cutoff": 1.65, "location": "Moratuwa" }],
    "private": [{ "name": "SLIIT", "degree": "Computer Science",
        "tuition_fee": "LKR 500,000/year", "location": "Malabe" }]
  },
  "best_degree": "Computer Science",
  "confidence": 0.89,
  "method": "trained_random_forest"
}
```

Table A.2: Subject-stream mapping

Stream	Core subjects	Optional subjects
Physical Science	Combined Mathematics, Physics, Chemistry	Applied Mathematics, Further Mathematics, Statistics, ICT
Biological Science	Biology, Chemistry, Physics	Agricultural Science, Environmental Science, ICT
Commerce	Accounting, Business Studies, Economics	Statistics, Mathematics, ICT
Arts	Three from languages, civilisations, social sciences and aesthetics	-
Engineering Technology	Engineering Technology, Science for Technology, ICT	Electronics, Computer Science, Applied Mathematics
Bio Systems Technology	Bio Systems Technology, Science for Technology, ICT	Agricultural Science, Environmental Science, Electronics

Appendix B: Academic integrity and AI use disclosure

Generative AI tools were used in this dissertation in line with the module policy. All content was reviewed, verified and edited by the author, who remains responsible for its accuracy, originality and citations.

Table B.1: AI tool use disclosure

Tool	Purpose	Sections affected	Verification performed
[e.g. Claude]	Structuring and formatting of dissertation text	All chapters	Content checked against own work, sources and results
[e.g. GitHub Copilot]	Code suggestions	[e.g. Flask backend, React components]	Code reviewed and tested